

Document outline:

Installation	1
The package	1
The datasets	1
Neighborhood data	1
Neighborhood borders	1
Network, bus-stop locations, Points Of Interest	1
Gui / Dashboard	2
Python package	2
Simulation	2
Configuration	3
Example	3
Configurable parameters	3

Installation

The package

The package can be installed with pip with the following command:

```
pip install STDAP
```

The datasets

The simulation needs two datasets: The neighborhood data, and the neighborhood borders. The datasets can be downloaded from the Dutch Central Bureau of Statistics (CBS).

Neighborhood data

The dataset storing neighborhood data is called: “Kerncijfers wijken en buurten <year>”. Some columns are renamed across datasets of different years, and can thus be configured. The package uses the column names of the 2025 dataset by default. The datasets over the years can be found at the following link: “<https://www.cbs.nl/nl-nl/reeksen/publicatie/kerncijfers-wijken-en-buurten>”. This file should be converted to CSV format before its use.

Neighborhood borders

The dataset storing neighborhood borders is called: “Wijk- en buurtkaart <year>”. The column names of the 2025 dataset are used for this. Only the ‘buurtcode’ and ‘geom’ are read, differentiating datasets should have their columns renamed. The datasets over the years can be found at the following link: “<https://www.cbs.nl/nl-nl/dossier/nederland-regionaal/geografische-data>”. This file should be converted to geopackage format before its use.

Network, bus-stop locations, Points Of Interest

This data is all imported from OpenStreetMap using the OSMnx API. If specified, this data is written to the local file system. The simulation is capable of using these files instead to speed up simulations, and allow custom data to be imported. This is done by setting the `store_in_file` parameter to `True`, and setting the `storage_dir` parameter to the folder containing the files to write to / from. These parameters should be set when initializing the main Simulator class:

```
sim = Simulator(csv, geopackage, store_in_file=True,
               storage_dir="The_downloaded_graphs/")
```

The files are/should be called the following:

- The car-accessible network:
 - <city>_drive.graphml
- The pedestrian network:
 - <city>_ped.graphml
- The bus-stops and Points Of Interest
 - <city>.parquet

The datafiles are obtained using OSMnx features `_from_polygon` and `graph_from_polygon`.

Gui / Dashboard

The simulation dashboard can be started with the following Python code:

```
# Import gui
from STDAP.gui.dashboard import show_dashboard
# Run gui
show_dashboard()
```

Python package

Simulation

Below is a basic example, showing how to run a simulation. More examples can be found in the example folder within the package.

```
# The path's to the manually downloaded datasets
csv = "kwb2025.csv"
geopackage = "borders.gpkg"
# Initiate simulator with datasets, specify graphs should be downloaded to speed
up later simulations.
sim = Simulator(csv, geopackage, store_in_file=True,
                storage_dir="The_downloaded_graphs/")
# Extract all cities available for simulation
city_options = sim.get_cities()
# Choose the city you want to run simulations in.
# This should use the Dutch city names (see city_options).
sim.choose_city("Groningen")
# Simulate using 25% pedestrianization, completely redesigning the bus-network

sim.Sim_trans_dist_single(0.25,
                          svg=False,
                          bus_network_redesign=True,
                          minimal_move=False,
                          saving_dir="results_example/25%/")

# Simulate over a range of fractions to pedestrianize, move bus-stops to nearest
valid location.
sim.Sim_trans_dist_multiple(0,
                             0.5,
                             100,
                             svg=False,
                             bus_network_redesign=False,
```

```
minimal_move=True,
saving_dir="results_example/range/")
```

Configuration

Configuration can be done using the settings dataclass. Every configurable parameter is represented by a field in this dataclass. Fields containing can be treated like normal values, and can be configured by regular assignment. Most fields consist of values, however some fields contain functions. Fields containing values are fully typechecked, this was however not possible with all functions. When configuring fields containing functions, always read the description to understand the basic function requirements. Pre-defined functions can be imported separately to allow function parameters to be configured.

Example

The Python code below shows an basic configuration example. More examples can be found in the example folder in the package github page.

```
# Import the function needed to obtain the settings class
from STDAP.config.settings import get_settings
# Import the default neighborhood sampling function
from STDAP.config.functions import PoissonDiskDistribution

# Obtaining the settings class
settings = get_settings()
# Re-defining configurable parameters:
settings.min_distance_stops = 100
settings.max_distance_stops = 300
# Configuring the dataset column names and NULLstring to the format used in 2024
settings.dataset_column_names['high_education'] = "a_opl_bvm"
settings.dataset_column_names['medium_education'] = "a_opl_hvm"
settings.dataset_column_names['low_education'] = "a_opl_hw"
settings.dataset_nullstring = ['.', '.', '']
# Configuring the radius used by the sampling function
settings.neighborhood_distribution = (
    lambda a, b, c, d : PoissonDiskDistribution(a, b, c, d, radius=100)
)
```

Configurable parameters

Below is an overview of all configurable parameters. Such an overview containing current and default values is also obtainable using the settings.describe() and settings.to_df() functions of the settings class to obtain a string and Pandas DataFrame respectively.

Parameter	Type	Default		Description
dataset_column_names	Dictionary	Data content	Dataset column name	Maps internal STDAP field names to column names in the input datasets. Allows datasets from different years to be used without changing the simulation code. All referenced columns must exist. geom
		id	gwb_code	
		region	regio	
		city name	gm_naam	
		recs	recs	

Parameter	Type	Default		Description
		Data content	Dataset column name	and buurtcode are read from the geopackage; all other fields are read from the CSV.
		population	a_inw	
		count males	a_man	
		count females	a_vrouw	
		count age 0-14	a_00_14	
		count age 15-24	a_15_24	
		count age 25-44	a_25_44	
		count age 45-64	a_45_64	
		count age 65-oo	a_65_oo	
		count background nl	a_nl_all	
		count background eu	a_eur_al	
		count background neu	a_neu_al	
		count birthplace nl	a_geb_nl	
		count birthplace eu	a_geb_eu	
		count birthplace neu	a_geb_ne	
		count low education	a_opl_lg	
		count medium education	a_opl_md	
		count high education	a_opl_hg	

Parameter	Type	Default		Description
		Data content	Dataset column name	
		count low income	p_ink_li	
		count high income	p_ink_hi	
		count risk poverty	p_ink_ar	
		buurtcode	buurtcode	
		geom	geom	
dataset_nullstring	List[String]	".", ".", ":", ":""		Strings interpreted as NULL values when reading CSV files.
dataset_delim	String	,		Delimiter character used when reading CSV files.
dataset_decimal_separator	String	,		Decimal separator used when parsing floating-point values.
neighborhood_distribution	Callable	STDAP.config: PoissonDiskDistribution		Function used to generate representative population points within a neighborhood bounding box. Must return a NumPy array of [x, y] coordinates.
max_dist_transit_network	Integer	30		Maximum distance (m) allowed between a transit stop and the car or pedestrian network.
transit_max_pts_dist	Integer	30		Maximum distance (m) between a generated neighborhood point and the nearest pedestrian-network node.
transit_max_move_dist	Integer	200		Maximum distance (m) that a transit stop may be moved by the minimal-move relocation algorithm.
min_distance_stops	Integer	300		Minimum allowed spacing (m) between transit stops in the blank-slate relocation method.

Parameter	Type	Default	Description
max_distance_stops	Integer	800	Maximum allowed spacing (m) between transit stops in the blank-slate relocation method.
max_stops_in_bus_route	Integer	30	Maximum number of stops allowed in a generated bus route.
min_stops_in_bus_route	Integer	9	Minimum number of stops allowed in a generated bus route.
amenity_to_pop_weight	Float	20	Weight assigned to amenities in the blank-slate stop scoring formula: score = population + weight * amenities
png_dpi	Integer	500	Resolution used when exporting PNG visualizations.
colormap	String	Matplotlib RdBu_r	Matplotlib colormap used for network visualizations.
color_normalization	Callable	mcolor SymLogNorm Matplotlib with a linthresh of 1	Normalization function used when mapping values to colors in network visualizations.
legend_num_labels	Integer	10	Number of tick labels shown on the color-bar legend.